

1. Apply Terraform to create EC2

Terraform code:

1.variables.tf

```
variables.tf
1 variable "aws_region" { 2 usages
2   description = "AWS region to deploy resources in."
3   type       = string
4   default    = "us-east-1"
5 }
6
7 variable "project_name" { 6 usages
8   description = "A name prefix used for tagging and naming resources."
9   type       = string
10  default    = "static-website"
11 }
12
13 variable "instance_type" { 2 usages
14   description = "EC2 instance type."
15   type       = string
16   default    = "t2.micro" # Free-tier eligible
17 }
18
19 variable "key_pair_name" { 5 usages
20   description = "Name of an EXISTING EC2 key pair to enable SSH access. Leave empty to skip SSH key."
21   type       = string
22   default    = ""
23 }
24
25 variable "github_repo_url" { 2 usages
26   description = "HTTPS clone URL of the GitHub repository that contains the static website."
27   type       = string
28   # Example: https://github.com/your-username/your-static-site.git
29   default    = "https://github.com/cloudacademy/static-website-example.git"
30 }
31
32 variable "github_branch" { 2 usages
33   description = "Branch to pull from the GitHub repository."
34   type       = string
35   default    = "main"
36 }
37
```

2.main.tf

```
user_data.sh.tpl  outputs.tf  main.tf
1 locals {
2   # Render the bootstrap script with our variables injected.
3   user_data = templatefile("${path.module}/user_data.sh.tpl", { 1 usage
4     github_repo_url = var.github_repo_url
5     github_branch   = var.github_branch
6     site_subdir      = var.site_subdir
7   })
8 }
9
10 resource "aws_instance" "web" { 5 usages
11   ami           = data.aws_ami.amazon_linux.id
12   instance_type = var.instance_type
13   vpc_security_group_ids = [aws_security_group.web_sg.id]
14
15   # Key_pair_name may be empty; only set it if provided.
16   key_name = var.key_pair_name != "" ? var.key_pair_name : null
17
18   # Bootstrap: install Nginx, pull from GitHub, deploy.
19   user_data = local.user_data
20   user_data_replace_on_change = true
21
22   root_block_device {
23     volume_size = 8
24     volume_type = "gp3"
25   }
26
27   tags = {
28     Name     = "${var.project_name}-ec2"
29     Project = var.project_name
30   }
31 }
```

3.user_data.sh.tpl

```
user_data.sh.tpl x
Would you like to install shellcheck to verify your shell scripts? Install No
Possible template file Treat this file as a Terraform Template Dismiss
1 |> #!/bin/bash
2 | s -e -u -o pipefail
3 |
4 | # Log everything to a file for easy debugging on the instance
5 | exec >>(tee /var/log/user-data.log) 2>&1
6 |
7 | echo "===== Updating packages ====="
8 | dnf update -y
9 |
10 | echo "===== Installing Nginx and Git ====="
11 | dnf install -y nginx git
12 |
13 | echo "===== Enabling and starting Nginx ====="
14 | systemctl enable nginx
15 | systemctl start nginx
16 |
17 | echo "===== Cloning GitHub repository ====="
18 | APP_DIR="/opt/website"
19 | rm -rf "$APP_DIR"
20 | git clone --branch "${github_branch}" --depth 1 "${github_repo_url}" "$APP_DIR"
21 |
22 | echo "===== Deploying static files to Nginx web root ====="
23 | WEB_ROOT="/usr/share/nginx/html"
24 | rm -rf "${WEB_ROOT:?}/*"
25 |
26 | # Copy the contents of the chosen subdirectory into the web root
27 | cp -r "$APP_DIR/${site_subdir}/." "$WEB_ROOT/"
28 |
29 | # Make sure permissions are correct
30 | chown -R nginx:nginx "$WEB_ROOT"
31 | chmod -R 755 "$WEB_ROOT"
32 |
33 | echo "===== Restarting Nginx to serve new content ====="
34 | systemctl restart nginx
```

Output.tpl

```
terraform apply -auto-approve
data.aws_vpc.default: Reading...
data.aws_ami.amazon_linux: Reading...
data.aws_ami.amazon_linux: Read complete after 2s [id=ami-00948338a4aee604]
data.aws_vpc.default: Read complete after 3s [id=vpc-0f2eff95c5e4cbf95]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.web will be created
+ resource "aws_instance" "web" {
  + ami                  = "ami-00948338a4aee604"
  + arn                  = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone     = (known after apply)
  + cpu_core_count       = (known after apply)
  + cpu_threads_per_core  = (known after apply)
  + disable_api_stop      = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized         = (known after apply)
  + enable_primary_ipv6   = (known after apply)
  + get_password_data     = false
  + host_id               = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile  = (known after apply)
  + id                   = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle    = (known after apply)
  + instance_state        = (known after apply)
  + instance_type         = "t3.micro"
  + key_name              = "my-key"
  + monitoring              = false
  + network_interface_ids = []
  + placement_group        = ""
  + platform               = "Linux"
  + private_ip             = (known after apply)
  + public_ip              = (known after apply)
  + subnet_id              = "subnet-0a1b2c3d"
  + tags                   = {}
  + tenancy                = "default"
  + time_zone              = ""
  + user_data              = "user_data.sh.tpl"
  + vpc_security_group_ids = ["sg-0a1b2c3d"]
}
```

This is the public IP of the Instance

```
Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ instance_id = (known after apply)
+ public_dns = (known after apply)
+ public_ip   = (known after apply)
+ ssh_command = "No key pair configured."
+ instance_id = (known after apply)
+ public_ip   = (known after apply)
+ ssh_command = "No key pair configured."
+ website_url = (known after apply)
aws_security_group.web_sg: Creating...
aws_security_group.web_sg: Creation complete after 5s [id=sg-09295d47628ac3f4f]
aws_instance.web: Creating...
aws_instance.web: Still creating... [00m12s elapsed]
aws_instance.web: Still creating... [00m22s elapsed]
aws_instance.web: Still creating... [00m32s elapsed]
aws_instance.web: Creation complete after 38s [id=i-0a09e644e5f3585ca]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
Outputs:
instance_id = "i-0a09e644e5f3585ca"
public_dns = "ec2-54-82-113-216.compute-1.amazonaws.com"
public_ip = "54.82.113.216"
ssh_command = "No key pair configured."
website_url = "http://54.82.113.216"
```

Result of EC2 created by Terraform

Instances (1/1) Info

Last updated 2 minutes ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

< 1 >

Settings

☒	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone
☒	static-website-ec2	i-0a09e644e5f3585ca	Running	t2.micro	2/2 checks passed	us-east-1d

Instance summary for i-0a09e644e5f3585ca (static-website-ec2) Info

Updated 2 minutes ago

Connect

Instance state

Actions

Instance ID

i-0a09e644e5f3585ca

IPV6 address

-

Hostname type

IP name: ip-172-31-28-130.ec2.internal

Answer private resource DNS name

-

Auto-assigned IP address

Failed to retrieve Auto-assigned IP address

IAM role

-

IMDSv2

Required

Operator

-

Public IPv4 address

54.82.113.216 | open address

Instance state

Running

Private IP DNS name (IPv4 only)

ip-172-31-28-130.ec2.internal

Instance type

t2.micro

VPC ID

vpc-0f2eff95c5e4cbf95

Subnet ID

subnet-014b03e118aef6373

Instance ARN

arn:aws:ec2:us-east-1:158409804400:instance/i-0a09e644e5f3585ca

Private IPv4 addresses

172.31.28.130

Public DNS

ec2-54-82-113-216.compute-1.amazonaws.com | open address

Elastic IP addresses

Failed to retrieve Elastic IP addresses

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Auto Scaling Group name

-

Managed

false

Access to the Nginx with public IP "http://54.82.113.216"

